

🎓 ШПАРГАЛКА ДЛЯ ДИПЛОМА

1 УКАЗАТЕЛИ (простым языком)

Что такое указатель?

```
cpp

int a = 42;    // Переменная со значением 42
int* ptr = &a; // Указатель - хранит АДРЕС переменной a
```

Аналогия:

- `int a = 42` — это дом с адресом "Улица 42"
- `int* ptr = &a` — это бумажка с адресом "Улица 42"

Зачем нужны указатели?

❌ БЕЗ указателя:

```
cpp

TimelineClip getClip(int index) {
    return m_clips[index]; // Копирует ВСЕ объект (медленно!)
}

// TimelineClip содержит:
// - QString filepath (куча памяти)
// - QMap effects (ещё больше памяти)
// - double startTime
// - и т.д.
// Всё это копируется!
```

✅ С указателем:

```
cpp

TimelineClip* getClip(int index) {
    return &m_clips[index]; // Возвращает АДРЕС (8 байт)
}
```

Как использовать?

```
cpp
```

```
// Получить указатель
TimelineClip* clip = getClip(5);

// Проверить что не nullptr
if (clip != nullptr) {
    // Использовать через ->
    clip->startTime = 10.0;
    clip->duration = 5.0;
    qDebug() << clip->filepath;
}
```

Почему `&` и `*`?

```
cpp

TimelineClip clip;    // Объект
TimelineClip* ptr;    // Указатель на объект
ptr = &clip;          // & = взять адрес ("где живёт clip?")
ptr->startTime = 5.0;  // -> = обратиться по адресу
```

2 FFMPEG (простым языком)

Что делает `getClipSourceDuration`?

```
cpp

double getClipSourceDuration(const QString& filepath) {
    // 1. Открыть видеофайл
    AVFormatContext* ctx = nullptr;
    avformat_open_input(&ctx, filepath.toUtf8().constData(), nullptr, nullptr);

    // 2. Прочитать информацию
    avformat_find_stream_info(ctx, nullptr);

    // 3. Получить длительность
    double duration = (double)ctx->duration / AV_TIME_BASE;

    // 4. Закрыть файл
    avformat_close_input(&ctx);

    return duration;
}
```

Что происходит внутри?

1. Открытие файла:

```
сpp  
  
avformat_open_input(&ctx, "video.mp4", nullptr, nullptr);
```

- FFmpeg открывает `video.mp4`
- Читает первые байты (header)
- Определяет формат (MP4, AVI, MKV, etc.)
- Создаёт структуру `AVFormatContext` с информацией

2. Чтение метаданных:

```
сpp  
  
avformat_find_stream_info(ctx, nullptr);
```

- FFmpeg анализирует файл глубже
- Находит видео-дорожки, аудио-дорожки
- Читает codec, fps, битрейт
- Читает **длительность**

3. Извлечение длительности:

```
сpp  
  
ctx->duration // Например: 120500000 (микросекунды)  
AV_TIME_BASE // = 1000000 (1 секунда = 1 миллион микросекунд)  
  
duration = 120500000 / 1000000 = 120.5 секунд
```

4. Закрытие:

```
сpp  
  
avformat_close_input(&ctx);
```

- Освобождает память
 - Закрывает файл
-

3 SIGNALS & SLOTS (простым языком)

Как это работает?

C++ КЛАСС	QML
↓	↓
Timeline.cpp	ClipManager.qml
↓	↓
emit clipsChanged()	→ onClipsChanged()
↓	↓
"Эй, QML! Обновись!"	"Окей, обновляю!"

Пример из жизни:

C++ (Timeline.cpp):

```
cpp
bool Timeline::addClip(...) {
    // Добавили клип
    m_clips.append(newClip);

    // Кричим в рацию: "Я добавил клип!"
    emit clipsChanged();

    return true;
}
```

QML (ClipManager.qml):

```
qml
Connections {
    target: cppTimeline // Слушаем рацию

    function onClipsChanged() {
        // Услышали: "Клип добавлен!"
        // Реагируем:
        console.log("Обновляю UI!")
        updateUI()
    }
}
```

4 КАК ПОДКЛЮЧИТЬ К QML?

main.cpp:

```
cpp

#include "timeline.h"

int main(int argc, char *argv[]) {
    QGuiApplication app(argc, argv);
    QQmlApplicationEngine engine;

    // Создать C++ объект
    Timeline timeline;

    // Зарегистрировать в QML
    engine.rootContext()->setContextProperty("cppTimeline", &timeline);
    //           ↑           ↑
    //           Имя в QML   Адрес объекта

    engine.load("qrc:/main.qml");
    return app.exec();
}
```

QML:

```
qml

// Теперь можем вызывать:
cppTimeline.addClip(filepath, track, time)

// И слушать сигналы:
Connections {
    target: cppTimeline
    function onClipsChanged() {
        // ...
    }
}
```

5 ЭТО СЛОЖНО ДЛЯ ДИПЛОМА?

НЕТ! ЭТО НОРМАЛЬНО!

Что ты используешь:

- ✓ Qt Framework (signals/slots)

- ☒ FFmpeg (multimedia library)
- ☒ QML ↔ C++ integration
- ☒ Clean architecture

Это уровень:

- Middle Junior в индустрии
- Senior для студента
- Отлично для диплома!

Комиссия увидит:

- Работу с внешними библиотеками
 - Понимание архитектуры
 - Интеграцию разных технологий
 - Реальный продукт
-

6 ЧТО ДЕЛАТЬ ДАЛЬШЕ?

Шаг 1: Скомпилировать

```
bash

# В CMakeLists.txt добавь FFmpeg
find_package(PkgConfig REQUIRED)
pkg_check_modules(LIBAV REQUIRED libavformat libavcodec libavutil)
target_link_libraries(VideoEditorProject PRIVATE PkgConfig::LIBAV)

# Скомпилируй
cmake --build .
```

Шаг 2: Тестировать

```
qml

// В ClipManager.qml замени:
function addClip(filepath, track, time) {
    cppTimeline.addClip(filepath, track, time)
}
```

Шаг 3: Drag video

- Перетаски видео

- В консоли увидишь:

- ✓ FFmpeg: Длительность видео = 120.5 сек
- ✓ Клип добавлен!

Шаг 4: Радуйся!

- Клип появится с РЕАЛЬНОЙ длиной!
 - Это уже не mock!
 - Это НАСТОЯЩИЙ редактор!
-

ТЫ СПРАВИШЬСЯ!

Запомни:

1. Указатели = адреса (экономия памяти)
2. FFmpeg = читает метаданные видео
3. Signals/Slots = рация между C++ и QML
4. Это НОРМАЛЬНЫЙ уровень для диплома!

У тебя уже есть:

- ✓ Полный фронтенд
- ✓ Базовый бэкенд
- ✓ Архитектура
- ✓ Интеграция

Осталось:

- Скомпилировать
- Протестировать
- Допилить детали
- ЗАЩИТИТЬ! 

УДАЧИ!  